



ASSIGNMENT REPORT

Digital Design

SYS_TOP — UART-Controlled Register File & ALU System

Course	Digital Design
Assignment	Final System Project — RTL, Verification, Synthesis, DFT, Formality & CDC
Student Name	Mohamed Hossam El-Sawy
Instructor / TA	Eng.Ali Tamsah
Submission Date	October 2026

01 OBJECTIVE

This report documents the complete design flow of **SYS_TOP**, a UART-controlled digital system that receives command frames from a master (the testbench), executes register-file and ALU operations, and returns the results over UART. It covers the architecture, RTL implementation, self-checking simulation with waveform analysis, logic synthesis, scan (DFT) insertion, formal equivalence checking (Formality) and static lint/CDC sign-off (SpyGlass).

Reports and logs from the tool runs are condensed to their summaries; full raw reports are not reproduced.

02 PROBLEM STATEMENT / REQUIREMENTS

The system must execute commands received serially on **RX_IN** and send any result back on **TX_OUT**. Two asynchronous clock domains are involved, so the design must also handle clock-domain crossing (CDC) and reset synchronization safely.

System specifications

- Reference clock **REF_CLK** = **50 MHz** (period 20 ns); UART clock **UART_CLK** = **3.6864 MHz** (period $\approx$ 271.3 ns).
- The clock divider is always enabled (**clk_div_en** = 1).
- Top-level ports: inputs **REF_CLK**, **UART_CLK**, **RST** (active-low, asynchronous), **RX_IN**; outputs **TX_OUT**, **RF_PAR_ERR**, **RF_STP_ERR**.
- Ten functional blocks: RegFile, ALU, Clock Gating, **SYS_CTRL** (**REF_CLK** domain); **UART_TX**, **UART_RX**, **PULSE_GEN**, Clock Dividers (**UART_CLK** domain); **RST** synchronizers, Data synchronizer and **ASYNC FIFO** (crossings).
- Start-up sequence: configure UART through register-file writes to addresses 0x2 and 0x3, then the master sends RegFile / ALU commands; **SYS_CTRL** executes them and returns results via **UART_TX**.
- Addresses 0x0–0x3 are reserved for ALU operands and configuration; the spec describes the remaining range for normal read/write operations as starting at 0x4.

Supported commands

Command	Opcode	Frame sequence
RF Write (3 frames)	0xAA	Frame 0 = 0xAA, Frame 1 = address, Frame 2 = write data
RF Read (2 frames)	0xBB	Frame 0 = 0xBB, Frame 1 = address → 1 byte returned
ALU with operands (4 frames)	0xCC	Frame 0 = 0xCC, Frame 1 = operand A, Frame 2 = operand B, Frame 3 = ALU_FUN → 2 bytes returned (LSB, MSB)
ALU without operands (2 frames)	0xDD	Frame 0 = 0xDD, Frame 1 = ALU_FUN (uses the A/B already in REG0/REG1) → 2 bytes returned

Table 1: Command set. Each UART frame = 1 start bit, 8 data bits (LSB first), parity bit, 1 stop bit (11 bits).

Configuration registers

Register	Address	Function	Reset value
REG0	0x0	ALU operand A	0x00
REG1	0x1	ALU operand B	0x00
REG2	0x2	UART config: [0] parity enable, [1] parity type (0 = even), [7:2] prescale	0x81 (parity on, even, prescale 32)
REG3	0x3	Clock-divider ratio for the TX clock	0x20 ($\div 32$)

Table 2: Reserved register-file locations (reset values taken from the RegFile RTL).

03 DESIGN & ANALYSIS

3.1 Design Approach

The design is partitioned by clock domain, with every crossing handled by a dedicated synchronizer structure:

- **Reset:** one 2-flop reset synchronizer per domain (`RST_SYNC_1` on `REF_CLK`, `RST_SYNC_2` on `UART_CLK`) gives asynchronous assertion and synchronous de-assertion.
- **RX path (`UART_CLK` $\rightarrow$ `REF_CLK`):** the received byte and its valid flag cross through `DATA_SYNC` (2-flop synchronizer on the enable with edge-detect pulse generation that qualifies the bus).
- **TX path (`REF_CLK` $\rightarrow$ `UART_CLK`):** `SYS_CTRL` pushes result bytes into an 8-entry **asynchronous FIFO** with Gray-coded pointers and 2-flop pointer synchronizers. `PULSE_GEN` converts the `UART_TX busy` level into a one-cycle `R_INC` read pulse.
- **Static configuration:** `REG2` (parity/prescale) and `REG3` (divider ratio) are written once at start-up and then held, so they connect directly into the UART domain as quasi-static signals.
- **Baud-rate generation:** the TX divider divides `UART_CLK` by `REG3` (32), giving $3.6864 \text{ MHz} / 32 = 115.2 \text{ kHz}$ (bit period $8.68 \mu\text{s}$). The RX divider ratio is decoded from the one-hot prescale field by `CLKDIV_MUX` ($32 \rightarrow 1$, $16 \rightarrow 2$, $8 \rightarrow 4$, $4 \rightarrow 8$) so RX always samples at a fixed multiple of the baud rate; RX uses a 3-sample majority vote per bit.
- **Power:** the ALU clock is gated by a latch-based clock gate. `SYS_CTRL` enables it only while an ALU operation is in progress and removes it once `OUT_VALID` is seen.

3.2 State Table

`SYS_CTRL` is a 14-state FSM clocked by `REF_CLK`. The first received byte selects the command; later frames are captured in `frame1/2/3_reg`. Control outputs are one-cycle pulses.

State	Code	Behaviour
<code>S_IDLE</code>	0	Wait for <code>RX_D_VLD</code> ; latch opcode into <code>cmd_reg</code>
<code>S_FRAME1</code>	1	Capture frame 1; RF-Read $\rightarrow$ <code>S_RF_READ_REQ</code> , ALU-NOP $\rightarrow$ <code>S_ALU_EXEC</code> , else <code>S_FRAME2</code> ; unknown opcode $\rightarrow$ <code>IDLE</code>
<code>S_FRAME2</code>	2	Capture frame 2; RF-Write $\rightarrow$ <code>S_RF_WRITE</code> , ALU-OP $\rightarrow$ <code>S_FRAME3</code>
<code>S_FRAME3</code>	3	Capture ALU_FUN frame $\rightarrow$ <code>S_ALU_WR_A</code>
<code>S_RF_WRITE</code>	4	Pulse <code>WrEn</code> with <code>Address = frame1[3:0]</code> , <code>WrData = frame2</code>
<code>S_RF_READ_REQ</code>	5	Pulse <code>RdEn</code> with the requested address

State	Code	Behaviour
S_RF_READ_WAIT	6	Wait for RdData_Valid
S_RF_READ_SEND	7	If FIFO not full: push RdData to FIFO (TX_D_VLD)
S_ALU_WR_A / WR_B	8 / 9	Write operands into REG0 (addr 0) and REG1 (addr 1); enable the gated clock
S_ALU_EXEC	10	Assert EN, drive ALU_FUN (frame 3 for 0xCC, frame 1 for 0xDD)
S_ALU_WAIT	11	Wait for OUT_Valid, then drop CLK_EN
S_ALU_SEND_LSB / MSB	12 / 13	Push ALU_OUT[7:0] then ALU_OUT[15:8] to the FIFO when not full

Table 3: SYS_CTRL state table (from the RTL).

The ALU is registered, 16-bit wide at its output, and selected by a 4-bit ALU_FUN:

FUN	Operation	FUN	Operation	FUN	Operation
0000	$A + B$	0110	$\sim(A \& B)$	1100	$A < B \rightarrow 3$
0001	$A - B$	0111	$\sim(A B)$	1101	$A \gg 1$
0010	$A \times B$	1000	$A \wedge B$	1110	$A \ll 1$
0011	$A \div B$	1001	$A \sim\wedge B$	—	—
0100	$A \& B$	1010	$A == B \rightarrow 1$	—	—
0101	$A B$	1011	$A > B \rightarrow 2$	—	—

Table 4: ALU function encoding (from ALU.v). The $A < B$ comparison is implemented in addition to the operations listed in the spec.

3.3 Circuit / Block Diagram

Figure 1 shows the system architecture from the specification; Figure 2 shows the SYS_TOP symbol as seen after synthesis.

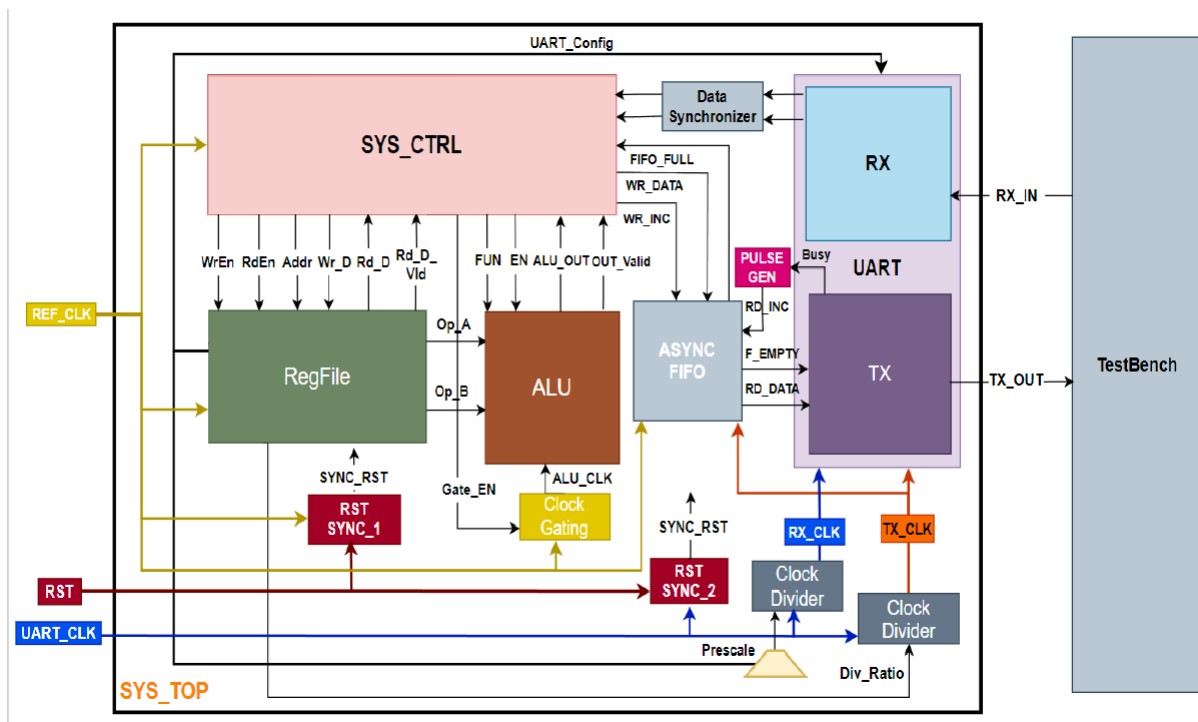


Figure 1: SYS_TOP block diagram — two clock domains, reset synchronizers, data synchronizer and asynchronous FIFO.



Figure 2: SYS_TOP top-level interface (REF_CLK, UART_CLK, RST, RX_IN → TX_OUT, RF_PAR_ERR, RF_STP_ERR).

Block	Domain	Role
RegFile	REF_CLK	16 × 8-bit register file; REG0–REG3 exposed as dedicated outputs
ALU	ALU_CLK (gated REF_CLK)	15 arithmetic/logic/compare/shift functions, registered 16-bit result + valid
CLK_GATE	REF_CLK	Latch + AND clock gate controlled by SYS_CTRL CLK_EN
SYS_CTRL	REF_CLK	Command decoder / FSM linking UART, RegFile, ALU and FIFO
UART_RX / UART_TX	RX_CLK / TX_CLK	Serial receiver (majority-vote sampling, parity & stop checks) and transmitter (serializer, parity calculation)
ClkDiv ×2, CLKDIV_MUX	UART_CLK	TX divider (REG3) and RX divider (decoded from prescale)
PULSE_GEN	TX_CLK	Rising-edge detector turning UART <i>busy</i> into FIFO read increment
RST_SYNC ×2, DATA_SYNC, ASYNC_FIFO	Crossings	Reset synchronization, RX bus crossing, TX data crossing

Table 5: Block summary.

3.4 HDL Implementation

The design is written in Verilog with SystemVerilog FSMs for the UART blocks (about 30 source files). The top level `SYS_TOP.v` instantiates every block and is parameterized (`RF_WIDTH=8`, `RF_DEPTH=16`, `RF_ADDR=4`, `FIFO_WIDTH=8`). Key excerpts follow.

TX crossing: FIFO + pulse generator (SYS_TOP.v)

```
PULSE_GEN U_PULSE_GEN (
    .clk(TX_CLK), .rst(SYNC_RST_2),
    .lvl_sig(UART_TX_BUSY), .pulse_sig(FIFO_R_INC)
);

ASYNC_FIFO #(.DATA_WIDTH(FIFO_WIDTH)) U_ASYNC_FIFO (
    .W_CLK(REF_CLK), .W_RST(SYNC_RST_1), .W_INC(SYS_TX_D_VLD),
    .R_CLK(TX_CLK), .R_RST(SYNC_RST_2), .R_INC(FIFO_R_INC),
    .WR_DATA(SYS_TX_P_DATA), .FULL(FIFO_FULL),
    .EMPTY(FIFO_EMPTY), .RD_DATA(FIFO_RD_DATA)
);
```

Clock gate behavioural model (CLK_GATE.v)

```
always @(CLK or CLK_EN)
    if (!CLK) Latch_Out <= CLK_EN; // transparent while CLK is low
assign GATED_CLK = CLK && Latch_Out; // glitch-free gated clock
```

For synthesis, this model is replaced by the library integrated clock-gating cell (`TLATNCAX12M`), which appears as a latch compare point in the Formality results.

Pulse generator (PULSE_GEN.v)

```
always @(posedge clk or negedge rst)
    if (!rst) begin rcv_flop <= 0; pls_flop <= 0; end
    else begin rcv_flop <= lvl_sig; pls_flop <= rcv_flop; end
assign pulse_sig = rcv_flop && !pls_flop; // one-cycle pulse on rising edge
```

04 SIMULATION & RESULTS

4.1 Verification Approach

Verification uses a self-checking SystemVerilog testbench (`SYS_TOP_tb`) run in ModelSim. The testbench acts as the UART master and also monitors `TX_OUT`:

- **Clocks:** REF_CLK period 20 ns; UART_CLK period 271.267 ns; UART bit period 8680.55 ns (115.2 kbaud).
- **Reset:** `RST` held low for 200 ns then released, followed by 1 μ s of wait so the reset synchronizers de-assert before traffic starts.
- **Configuration over UART:** writes `REG2 = 0x81` (parity enabled, even, prescale 32) and `REG3 = 0x20` (divide by 32), as required by the spec sequence.
- **Driver:** `send_uart_frame` generates start bit, 8 data bits LSB-first, even parity (`^data`) and stop bit.
- **Monitor:** on each test trigger it waits for the falling edge of `TX_OUT` (with a 200-bit-period timeout), samples every bit mid-cell, and compares each received byte with the expected value. For the `RegFile` write case no TX data is expected, so it waits 50 bit periods and prints `REGFILE[5]`.
- **Scoreboard:** counts total/passed/failed cases and prints a summary.

4.2 Test Results

TC	Test	Frames sent	Expected TX	Observed	Result
1	RegFile write 0x55 $\rightarrow$ addr 0x05	0xAA, 0x05, 0x55	none (RF[5] = 0x55)	RF[5] = 0x55	PASS
2	RegFile read addr 0x05	0xBB, 0x05	0x55	0x55	PASS
3	ALU ADD with operands 0x20 + 0x05	0xCC, 0x20, 0x05, 0x00	0x25, 0x00	0x25, 0x00	PASS
4	ALU SUB, no operands (0x20 – 0x05)	0xDD, 0x01	0x1B, 0x00	0x1B, 0x00	PASS
5	ALU MUL with operands 0x12 $\times$ 0x10	0xCC, 0x12, 0x10, 0x02	0x20, 0x01	0x20, 0x01	PASS

Table 6: Test cases and results. TC4 reuses the operands left in `REG0/REG1` by TC3. Results are returned LSB first (0x0025, 0x001B, 0x0120).

```

VSIM 21> run -all
#
# === STARTING TESTBENCH ===
#
# --- TC#1: RegFile Write [0x55 -> Addr 0x05] ---
# [REGFILE[5]] = 0x55
# TC#1: PASSED
# --- TC#2: RegFile Read [Addr 0x05, Exp: 0x55] ---
# [MATCH] Byte 0: 0x55
# [REGFILE[5]] = 0x55
# TC#2: PASSED
# --- TC#3: ALU ADD [0x20 + 0x05 -> 0x0025] ---
# [MATCH] Byte 0: 0x25
# [MATCH] Byte 1: 0x0
# [REGFILE[5]] = 0x55
# TC#3: PASSED
# --- TC#4: ALU SUB [0x20 - 0x05 -> 0x001B] ---
# [MATCH] Byte 0: 0x1b
# [MATCH] Byte 1: 0x0
# [REGFILE[5]] = 0x55
# TC#4: PASSED
# --- TC#5: ALU MUL [0x12 * 0x10 -> 0x0120] ---
# [MATCH] Byte 0: 0x20
# [MATCH] Byte 1: 0x1
# [REGFILE[5]] = 0x55
# TC#5: PASSED
#
# === TESTBENCH SUMMARY ===
# Total: 5 | Passed: 5 | Failed: 0
# STATUS: ALL TESTS PASSED!

```

Figure 3: ModelSim transcript — Total 5, Passed 5, Failed 0, "ALL TESTS PASSED".

4.3 Waveform Analysis

How to read the waveforms

All five waveforms use the same signal groups. The time axis is in nanoseconds; one UART frame (11 bits) lasts $\approx 95.5 \mu\text{s}$, which is why activity is spread over hundreds of microseconds while the REF_CLK-domain pulses are only 20 ns wide.

Signal / group	What it shows
U_SYS_CTRL/state,next_state	FSM state. It steps IDLE (0) → FRAME1 (1) → FRAME2 (2) → FRAME3 (3) as frames arrive. Single-cycle states (e.g. S_RF_WRITE, ALU states) last 20 ns and are too narrow to resolve at this zoom.
test_case_id,test_label, total_tc,passed_tc,failed_tc	Testbench bookkeeping; <code>passed_tc</code> increments when a case finishes, <code>failed_tc</code> stays 0 throughout.
tc_start_time, expected_num_bytes, expected_bytes	Start time of the case and the bytes the monitor expects on TX.
U0_UART_RX/P_DATA,data_valid	Byte decoded by UART_RX (hexagons) and its one-cycle valid pulse — one pulse per received frame.
U0_UART_TX/txOut,busy	Serial transmit line (idle high, start bit low) and the TX busy flag, high for the whole frame.
U_ASYNC_FIFO/RAM,WR_DATA, EMPTY,FULL,RD_DATA	FIFO storage, the byte pushed by SYS_CTRL (non-zero for one REF_CLK cycle), the empty flag (low while data waits) and the byte read by UART_TX. <code>FULL</code> stays low.
U_RegFile/regArr[0..15]	Register contents; [2] = 0x81 and [3] = 0x20 confirm the UART configuration.

Table 7: Waveform signal legend.

TC1 — RegFile write (0x55 → address 0x05)

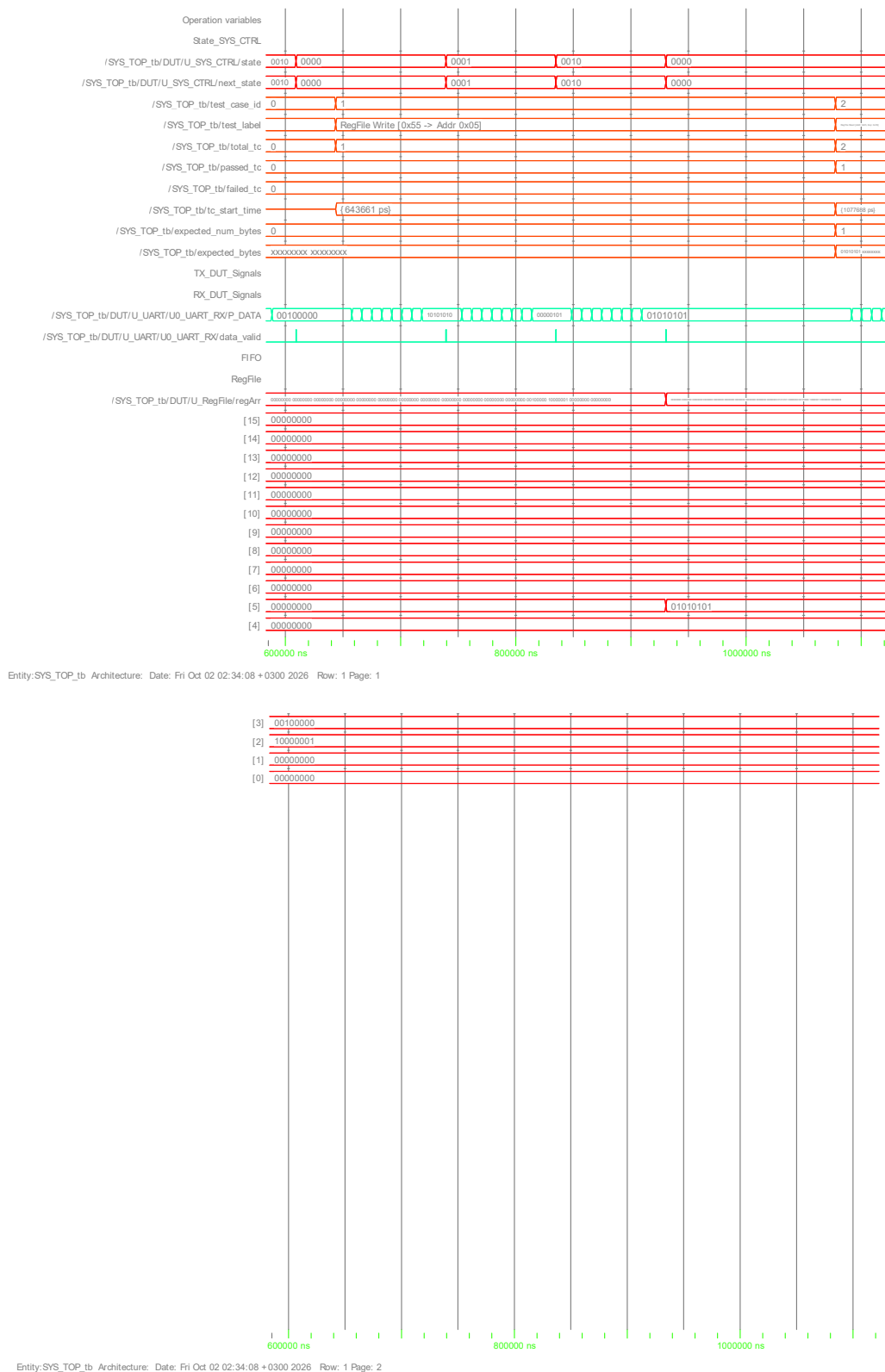
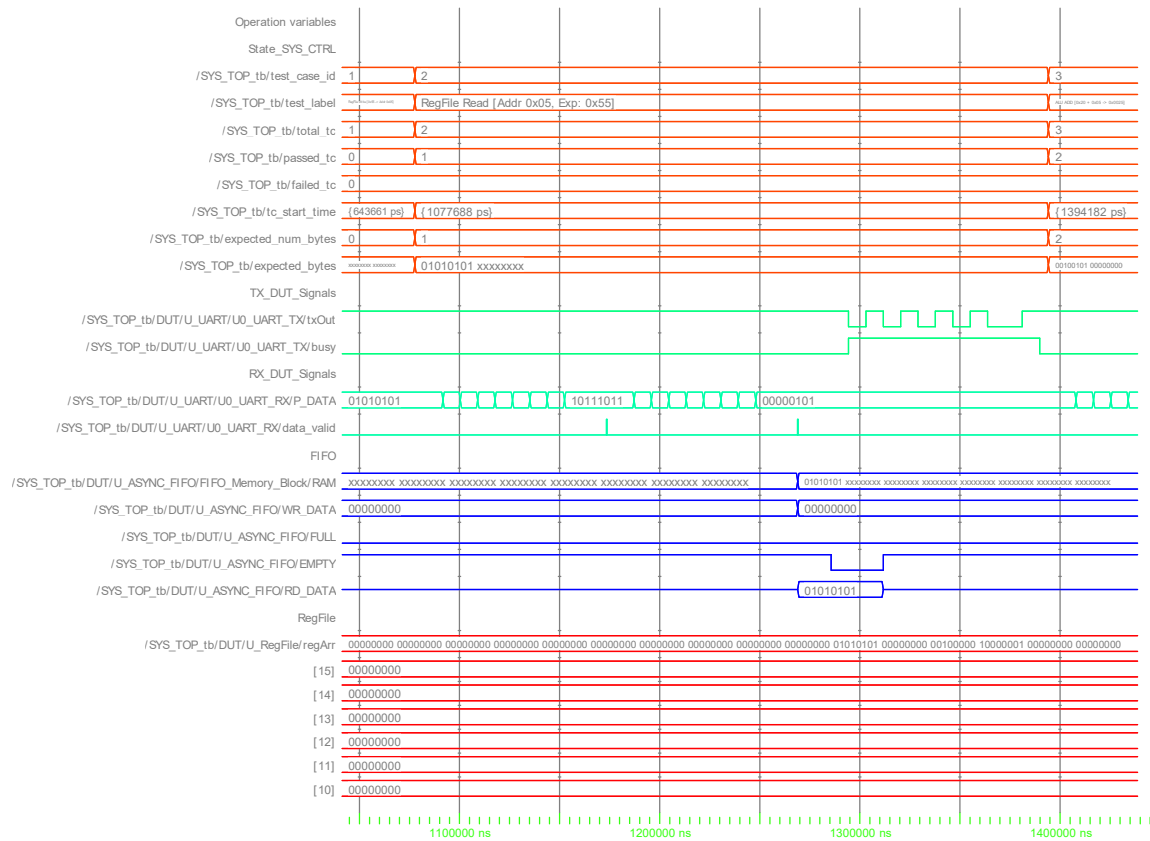


Figure 4: TC1 waveform, $\approx 0.60\text{--}1.10$ ms.

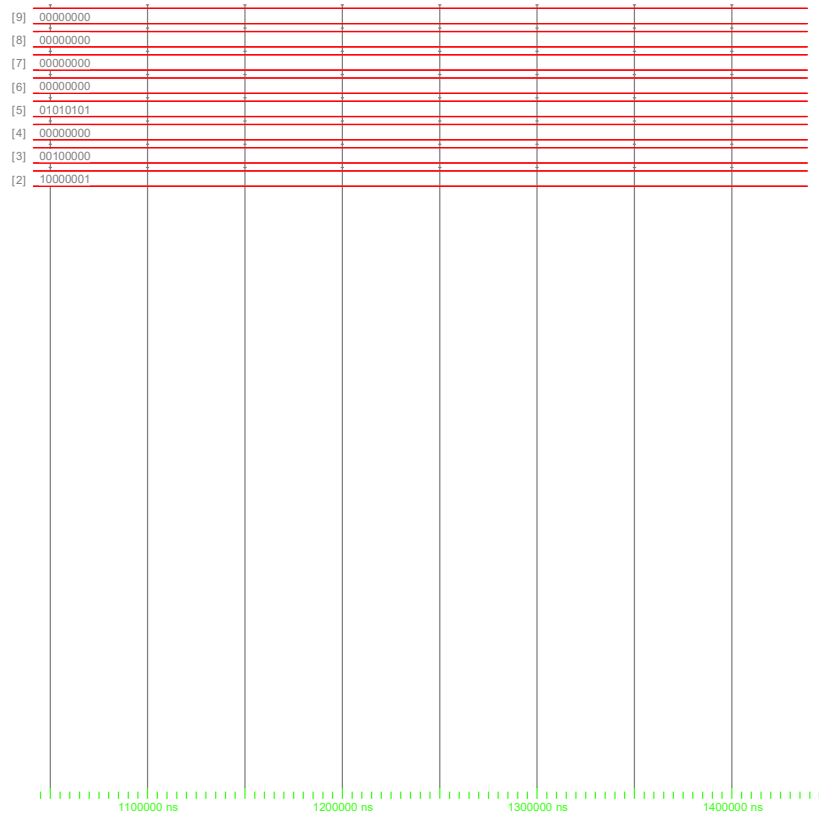
The case starts at $\approx 643.7 \mu\text{s}$. UART_RX decodes three frames — 0xAA (10101010), 0x05 (00000101) and 0x55 (01010101) — each marked by a `data_valid` pulse. SYS_CTRL moves 0000 $\rightarrow$ 0001 $\rightarrow$ 0010 as frames 0 and 1 arrive, returns to 0000 after the third frame, and `regArr[5]` changes from 0x00 to **0x55**.

at ≈ 0.93 ms. The leftmost frame (`00100000`) is the data byte of the earlier REG3 configuration write. No TX activity occurs, so the monitor waits 50 bit periods and the case ends at ≈ 1.078 ms (`passed_tc = 1`).

TC2 — RegFile read (address 0x05, expect 0x55)



Entity: SYS_TOP_tb Architecture: Date: Fri Oct 02 03:08:20 +0300 2026 Row: 1 Page: 1



Entity: SYS_TOP_tb Architecture: Date: Fri Oct 02 03:08:20 +0300 2026 Row: 1 Page: 2

Figure 5: TC2 waveform, $\approx 1.08\text{--}1.45\text{ ms}$.

Two frames arrive: 0xBB (10111011) and the address 0x05. SYS_CTRL reads the register and pushes 0x55 into the FIFO — RAM shows 01010101. EMPTY goes low, UART_TX reads the byte (RD_DATA = 01010101), busy rises and txOut sends one frame: start bit low, eight data bits, parity and stop. busy falls at the end of the frame, matching the expected byte 0x55.

TC3 — ALU ADD with operands (0x20 + 0x05)

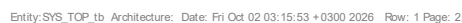
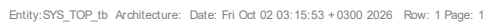


Figure 6: TC3 waveform, $\approx 1.39\text{--}2.00$ ms.

Four frames are decoded: 0xCC (11001100), 0x20 (00100000), 0x05 (00000101) and the function code 0x00 (ADD). The state trace shows 0000 → 0001 → 0010 → 0011 → 0000; the ALU states (write A, write B, execute, wait, send) complete within a few hundred nanoseconds after the fourth frame and are too narrow to resolve here. The FIFO receives two bytes — 00100101 (0x25) then 00000000 (0x00) — and `txOut` transmits two back-to-back frames while `EMPTY` stays low. The 16-bit result 0x0025 matches the expected value, and `regArr[0]/regArr[1]` now hold 0x20 and 0x05.

TC4 — ALU SUB without operands (0x20 – 0x05)

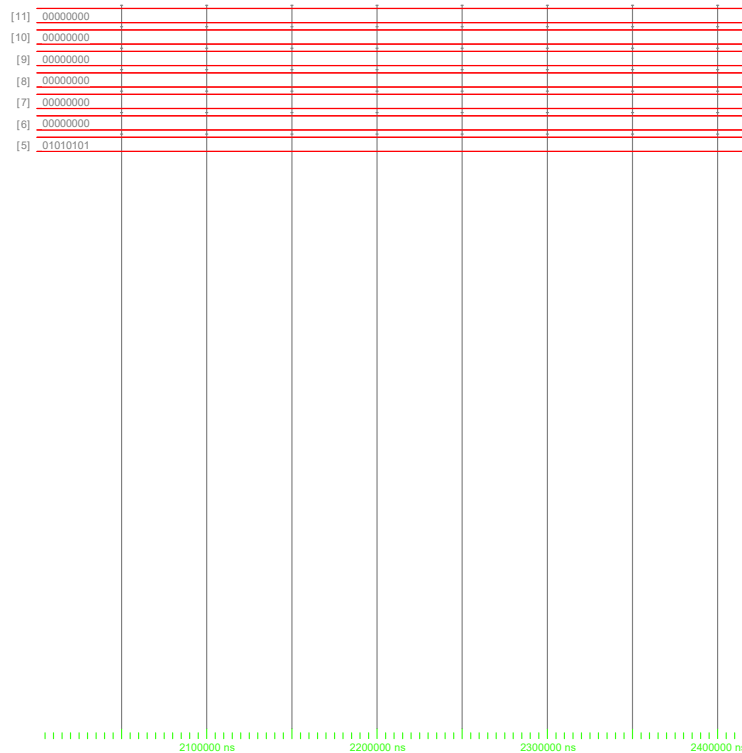
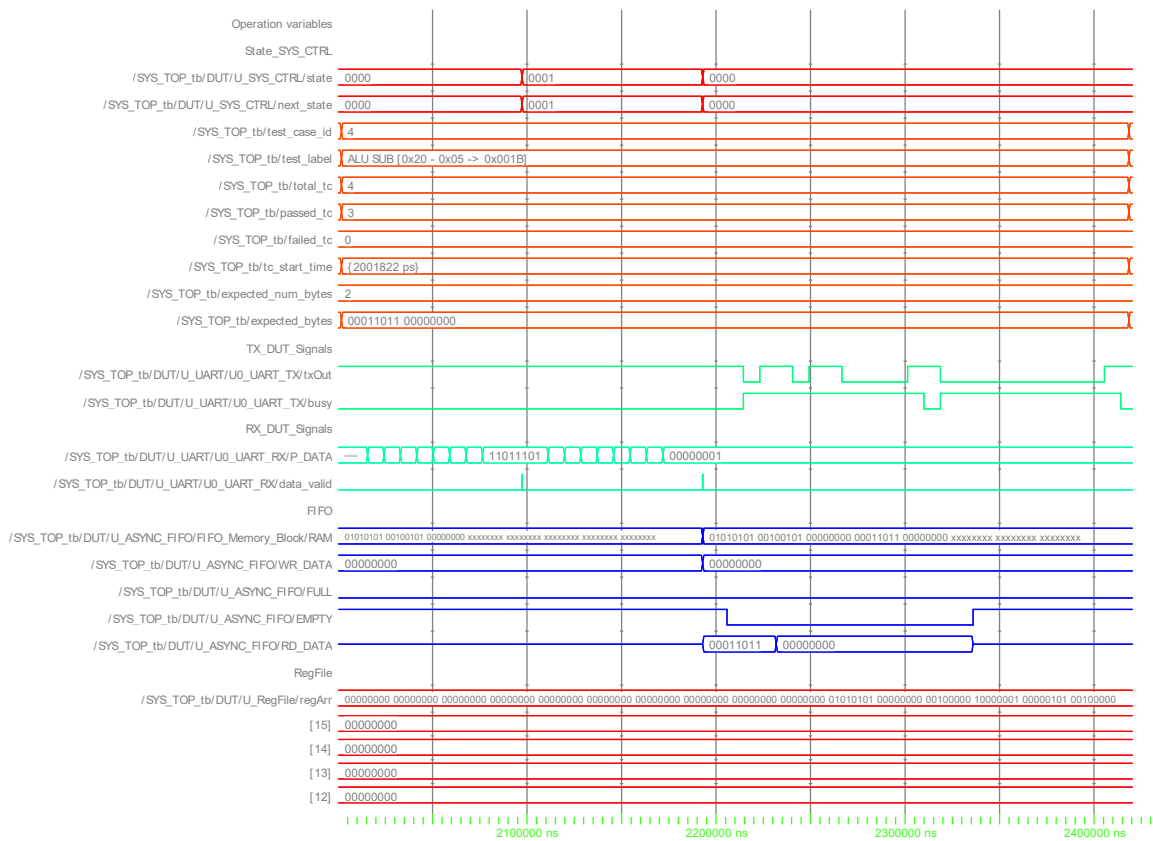
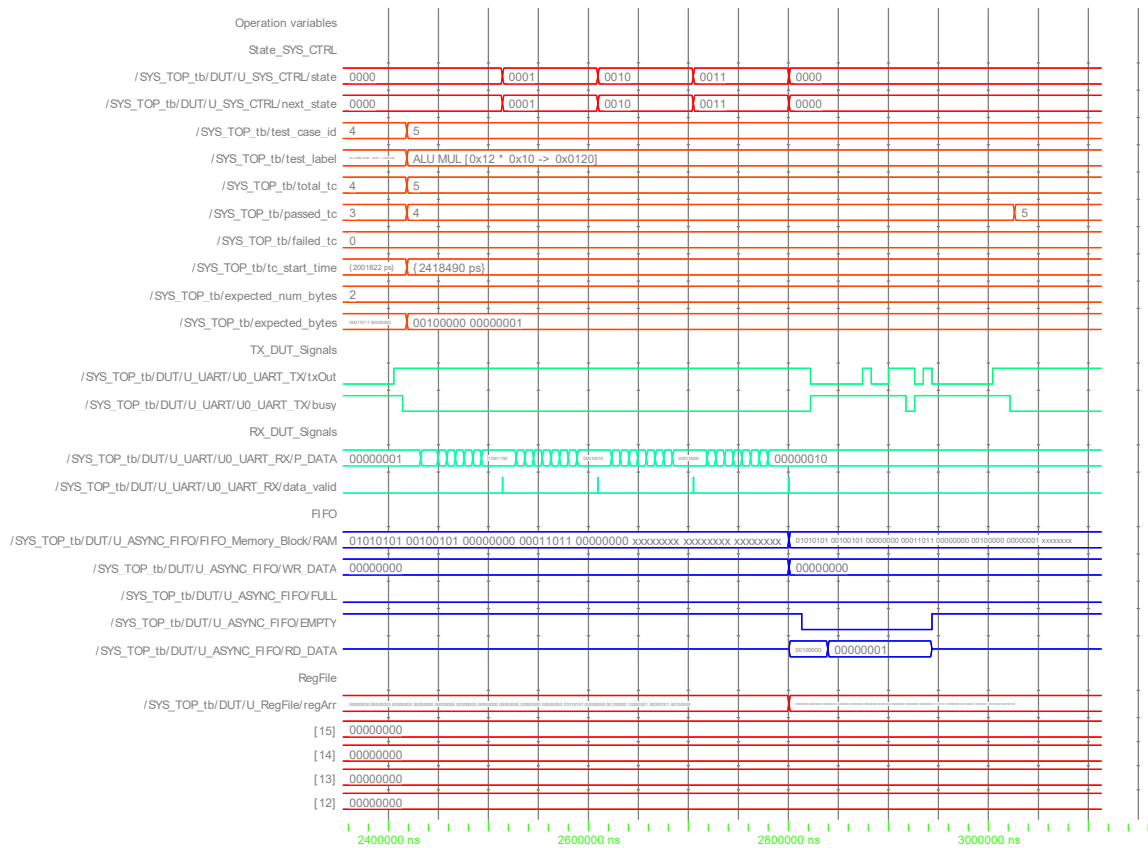


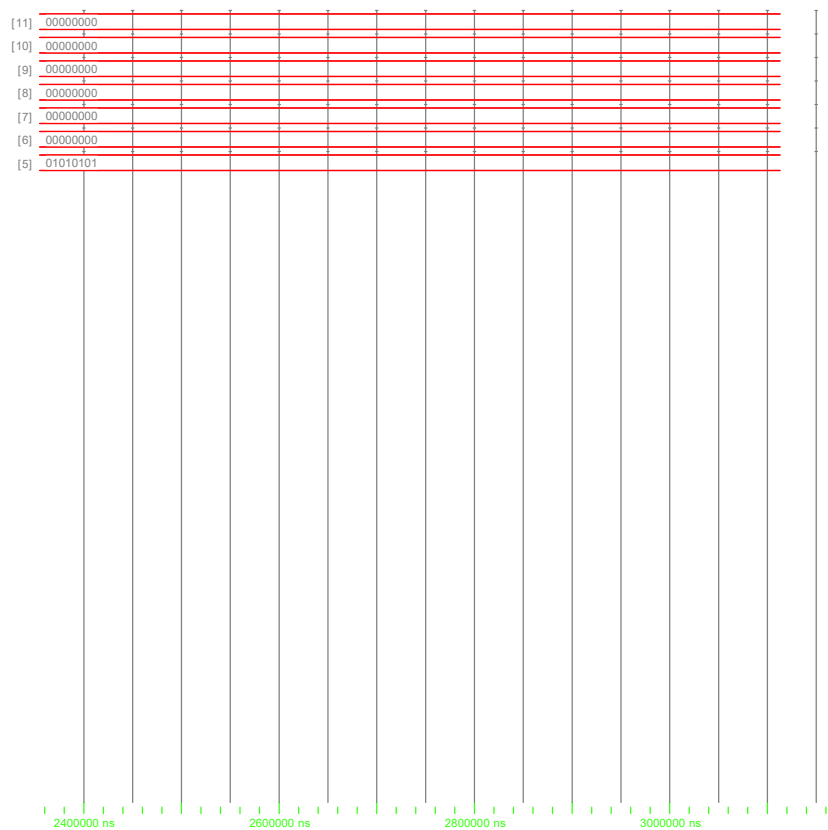
Figure 7: TC4 waveform, $\approx 2.00\text{--}2.45\text{ ms}$.

Only two frames are sent: 0xDD (**11011101**) and the function code 0x01 (SUB). The FSM goes 0000 → 0001 → 0000 because the no-operand command jumps from FRAME1 straight to execution, reusing REG0 = 0x20 and REG1 = 0x05 left by TC3 (visible in **regArr**). The FIFO holds **00011011** (0x1B) and **00000000**, and both bytes appear on **txOut** — result 0x001B as expected.

TC5 — ALU MUL with operands (0x12 × 0x10)



Entity:SYS_TOP_tb Architecture: Date: Fri Oct 02 03:20:27 +0300 2026 Row: 1 Page: 1



Entity:SYS_TOP_tb Architecture: Date: Fri Oct 02 03:20:27 +0300 2026 Row: 1 Page: 2

Figure 8: TC5 waveform, $\approx 2.40\text{--}3.10\text{ ms}$.

The four frames 0xCC, 0x12, 0x10 and 0x02 (MUL) produce the 16-bit product 0x0120, which exceeds one byte. SYS_CTRL therefore sends the low byte 00100000 (0x20) first and the high byte 00000001 (0x01) second, both visible in RD_DATA while EMPTY is low. The FIFO write pointer advances into new RAM locations (earlier results remain visible in the memory array) and FULL never asserts; passed_tc reaches 5 with failed_tc = 0.

TC	Start time	Trigger frames	Key evidence in waveform
1	$\approx 643.7\text{ }\mu\text{s}$	3 (RX)	regArr[5] = 0x55; no TX
2	$\approx 1077.7\text{ }\mu\text{s}$	2 (RX)	FIFO $\rightarrow$ RD_DATA 0x55 $\rightarrow$ 1 TX frame
3	$\approx 1394.2\text{ }\mu\text{s}$	4 (RX)	FIFO 0x25, 0x00 $\rightarrow$ 2 TX frames
4	$\approx 2001.8\text{ }\mu\text{s}$	2 (RX)	FIFO 0x1B, 0x00 $\rightarrow$ 2 TX frames
5	$\approx 2418.5\text{ }\mu\text{s}$	4 (RX)	FIFO 0x20, 0x01 $\rightarrow$ 2 TX frames

Table 8: Timeline of test cases (start times from the `tc\_start\_time` signal).

4.4 Synthesis Summary (Design Compiler)

Synthesis was run with Synopsys Design Compiler O-2018.06-SP1 using the TSMC 0.13 μm scmetro_tsmc_cl013g_rvt library at the slow corner (1.08 V, 125 $^{\circ}\text{C}$). Hold analysis in the report uses the fast corner (1.32 V, $-40\text{ }^{\circ}\text{C}$). The constraint report states: **"This design has no violated constraints."**

Metric	Value
Cells (total / combinational / sequential)	1945 / 1585 / 357
Buffers and inverters	259 cells (921.36 area units)
Combinational area	10970.37
Non-combinational (sequential) area	8947.63
Total cell area	19918.00
Hierarchical blocks kept	ClkDiv_RX 488.33 (2.5 %), ClkDiv_TX 643.65 (3.2 %), CLK_GATE 40.01 (0.2 %)
Total power (low-effort analysis)	0.227 mW (dynamic $\approx$ 0.217 mW, leakage $\approx$ 9.36 μW)
Constraint violations	None

Table 9: Synthesis area, power and constraint summary.

Path group	Check	Worst slack (ns)	Path	Result
ALU_CLK	Setup	10.06	RegFile regArr[1][5] → ALU ALU_OUT_reg[0] (arrival 9.39 ns)	MET
REF_CLK	Setup	14.92	SYS_CTRL state_reg[2] → RegFile RdData_reg[0]	MET
RX_CLK	Setup	215.83	RX stop-check → RF_STP_ERR output	MET
TX_CLK	Setup	6942.42	Serializer counter → TX_OUT	MET
UART_CLK	Setup	268.08	ClkDiv_TX counter [0] → [7]	MET
All groups	Hold	0.34 – 0.73	Tightest: RX data-sampling counter (0.34); loosest: SYS_CTRL → ALU_OUT_VALID_reg (0.73)	MET

Table 10: Worst setup/hold slack per clock group (one worst path reported per group).

Clock	Period (ns)	Source	Note
REF_CLK	20.00	Port	50 MHz
ALU_CLK	20.00	CLK_GATE/GATED_CLK	Generated, ÷1 of REF_CLK
UART_CLK	271.30	Port	3.6864 MHz
RX_CLK	271.30	ClkDiv_RX	Generated, ÷1 (prescale 32)
TX_CLK	8681.50	ClkDiv_TX	Generated, ÷32 → 115.2 kHz baud

Table 11: Clock definitions used in synthesis.

Log summary: compilation completed with no timing violations. The log contains 47 warnings, almost all lint-type notes: undriven-output cells (LINT-1) in ALU/ClkDiv/FIFO/UART blocks, constants on U_ALU and Mux_4X1 pins (LINT-32), clk_div_en tied to logic 1 (LINT-52), and CLKDIV_MUX outputs [7:4] tied to 0 (LINT-31/52). These match the design intent (divider always on, prescale decoded to ≤ 4 bits).

4.5 DFT Summary (Scan Insertion)

Scan insertion used a multiplexed-flip-flop style with full-scan methodology and no clock mixing (SCAN_CLK 20 ns, scan_rst active-low, test_mode and scan-enable SE active-high). The tool architected **4 scan chains** from the 355 sequential cells.

Chain	Scan ports	Cells	Starting element
1	SI[2] → SO[2]	89	RST_SYNC_1/Synchronizer_reg[0]
2	SI[1] → SO[1]	89	ASYNC_FIFO RAM_reg[5][7]
3	SI[0] → SO[0]	88	RegFile regArr_reg[3][4]
4	test_si4 → RF_STP_ERR	88	RegFile regArr_reg[14][4]

Table 12: Scan chain architecture.

DFT item	Result
Test design rule check	1 violation: TEST-505 — clock-gate latch U_CLK_GATE/U0_TLATNCAX12M has constant value 1 (1 of 355 sequential cells)
Valid scan cells	343 scan cells; 11 shift-register cells flagged non-scan by DRC
Fault list (uncollapsed stuck-at)	16 058 faults
Detected / ATPG-untestable / undetectable / not detected	15 939 / 81 / 36 / 2
Test coverage	99.48 %

Table 13: DFT DRC and coverage summary.

Synthesis vs. post-DFT netlist

Metric	Synthesis	After DFT	Change
Total cell area	19918.00	21876.03	+9.83 %
Combinational area	10970.37	10479.69	−4.47 %
Sequential area	8947.63	11396.34	+27.37 %
Cells	1945	1813	−6.8 %
Total power (mW)	0.227	0.387	+70 %
Setup, worst slack reported	10.06 ns (ALU_CLK)	13.02 ns (SCAN_CLK)	MET
Hold, worst slack reported	0.34 ns	0.32 ns (SCAN_CLK)	MET

Table 14: Effect of scan insertion. Sequential area grows because scan flip-flops include the input mux; combinational area shrinks as the original muxing is absorbed.

The added clock `SCAN_CLK` is defined at 20 ns alongside the functional clocks; all setup and hold checks in the DFT netlist are met.

4.6 Formal Equivalence Summary (Formality)

Synopsys Formality L-2016.03-SP1 compared the RTL (reference) against each netlist (implementation), with `synopsys_auto_setup` enabled.

Comparison	Passing points	Failing / aborted / unverified	Verification	Elapsed
RTL vs post-synthesis netlist	358	0 / 0 / 0	SUCCEEDED	≈ 137 s
RTL vs post-DFT netlist	358	0 / 0 / 0	SUCCEEDED	—

Table 15: Formality results.

Both comparisons matched 358 compare points: 3 ports, 354 flip-flops and 1 latch (the clock-gate latch). For the post-DFT run, `test_mode` and `SE` were set to constant 0 (functional mode) and the scan `SI/SO` ports were excluded from verification, so the scan-inserted netlist is functionally equivalent to the RTL in normal operation. Formality noted that auto-setup mode was used and RTL-interpretation messages occurred when linking the reference design, so results are valid under those settings.

4.7 SpyGlass Lint & CDC Summary

SpyGlass L-2016.06 was run in seven goals. Counts below are the "Results Summary" of each run; waived messages are listed separately.

Goal	Errors	Warnings	Info	Notes
Design_Read	0	0	7	SYS_TOP detected as the top unit; SDC → SGDC translation successful
lint_rtl	1	0	6	InferLatch in <code>CLK_GATE.v</code> line 24 (intended clock-gate latch); no combinational loops
cdc_setup_check	0 (1 waived)	0	14	Setup_port01 waived
cdc_clock_reset_integrity	0	0 (2 waived)	11	Clock_glitch04 ×2 waived
cdc_abstract	0 (1 waived)	0	12	Setup_port01 waived
cdc_verify_struct	0 (1 waived)	0 (1 waived)	40	Setup_port01, Ac_conv01 (+1 info) waived
cdc_verify	0 (1 waived)	0 (5 waived)	45	Full CDC verification — see below

Table 16: SpyGlass results by goal (reported messages after waivers).

CDC / reset metric (cdc_verify)	Result
Clocks / resets defined	2 clocks (REF_CLK, UART_CLK), 1 asynchronous reset
Registers / flat instances	367 / 1290
Unsynchronized scalar crossings (Ac_undef01)	0 (7 synchronized)
Unsynchronized vector crossings (Ac_undef02)	0 (14 synchronized)
Quasi-static crossings	15 (2 quasi-static constraints)
Reset synchronization	2 synchronized resets (RST_SYNC_1/2), both with synchronous de-assertion; 0 unsynchronized, 0 asynchronously de-asserted
Registers without clock / reset definition	0 / 0
Combinational loops	0
Black boxes	0

Table 17: CDC and reset-integrity summary.

Waived messages

Rule	Message (shortened)
Setup_port01	1 of 4 top-level ports (25 %) not associated with a clock constraint
Ac_clockperiod03	Design cycles between REF_CLK and UART_CLK exceed the analysis threshold
Ac_cdc01a	Fast-to-slow crossing from FIFO <code>wptr_inner[3]</code> into the <code>sync_w2r</code> synchronizer — data-hold check partially proved
Ac_conv01	5 synchronizers (FIFO r2w pointers and RX Data_Sync) converge on <code>SYS_CTRL.cmd_reg[7]</code>
Ac_datahold01a (x2)	FIFO RAM → UART_TX <code>parBit</code> and <code>pDataReg[7:0]</code> : data-enable sequencing partially proved
Clock_glitch04 (x2)	RegFile REG2/REG3 bits converge combinationally into the clock path of the UART FSM state flops

Table 18: Waived CDC/clock-reset messages.

05 DISCUSSION

Functional behaviour. All five test cases passed and the waveforms agree with the FSM design: frame counting in `SYS_CTRL`, FIFO hand-off to the TX domain, LSB-first transmission of the 16-bit ALU result, and operand reuse by the no-operand command. The 1.0 μ s post-reset wait in the testbench is needed because the reset synchronizer takes about 0.54 μ s (UART_CLK domain) to release.

Timing. Every path group meets setup with large margin. The tightest is the ALU (10.06 ns slack on a 20 ns period, i.e. a 9.39 ns data path), so the 50 MHz target is conservative for this library corner; the UART-domain paths have slack of hundreds of nanoseconds because the clocks are slow. Hold slack is positive everywhere (≥ 0.32 ns). Note that only the worst path per clock group is reported, and clock networks are ideal at this stage, so post-layout CTS may change these numbers.

Area and power trade-off. Scan insertion costs +9.83 % area (the flops become mux-D scan cells) and the reported power rises from 0.227 mW to 0.387 mW. In exchange the design reaches 99.48 % stuck-at test coverage. The power figures come from a low-effort analysis with default activity, and the scan netlist is analysed with the scan clock defined, so the comparison is indicative rather than a signoff number.

DFT limitations. The only DRC violation is TEST-505 on the clock-gate latch (constant 1 in test mode). The residual 119 untested faults (81 ATPG-untestable, 36 undetectable, 2 not detected) are 0.74 % of the fault list. A natural follow-up is to check that the gated-clock ALU flops are reachable in shift/capture mode, for example by forcing the gate enable in test mode.

Formal equivalence. Formality proves the synthesized and scan-inserted netlists equal to the RTL (358/358 points). The scan-inserted result is valid for functional mode only (`test_mode = 0`, `SE = 0`), which is the standard setup.

Lint and CDC. The single lint error is the latch inferred in the behavioural clock-gate model. It is intentional and is replaced by the library clock-gating cell during synthesis (Formality shows it as a latch compare point). CDC verification found **no unsynchronized crossings** (7 scalar and 14 vector crossings are all synchronized) and every reset is synchronized with synchronous de-assertion. The waived items are typical of an asynchronous-FIFO design — pointer and data-hold checks that SpyGlass can only partially prove — and static configuration registers that are written once at start-up. The waivers should be supported by a short justification in the project file; the unconstrained top-level port (Setup_port01) is worth closing by constraining it in the SGDC (the report does not name it here; the asynchronous `RX_IN` input is the usual candidate).

Points to note

- The spec text gives the normal-access address range as starting at 0x4; with the default 4-bit address bus the register file has 16 locations (0x0–0xF).
- The ALU implements an extra $A < B$ comparison (FUN = 1100) that is not in the spec list.
- Tools count registers differently (357 sequential cells in area reports, 355 in DFT DRC, 354 flops + 1 latch in Formality, 367 RTL registers in SpyGlass), so these totals are not expected to be identical.

06 CONCLUSION

- SYS_TOP was implemented per the specification, with two clock domains safely bridged by reset synchronizers, a data synchronizer and an 8-entry Gray-code asynchronous FIFO.
- The self-checking testbench passed **5/5** test cases covering register-file write/read and ALU add, subtract and multiply, and the waveforms confirm the expected frame-level behaviour.
- Synthesis produced a netlist of 19 918 area units with **no constraint violations** and positive setup/hold slack in all clock groups.
- Scan insertion added four chains for **99.48 %** test coverage at a +9.83 % area cost.
- Formality verified RTL $\equiv$ post-synthesis and RTL $\equiv$ post-DFT netlists (358 passing compare points, no failures).
- SpyGlass found no unsynchronized CDC paths; the only error is the intentional clock-gate latch, and a small set of waived warnings is documented above.